

Cahier des Charges

Présentation Générale et Objectifs

Contexte : Le Sound Design entre Échantillonnage et Synthèse

Dans l'industrie du jeu vidéo, du cinéma et de la musique, le *sound design* (conception sonore) consiste à bâtir l'identité sonore d'un univers (bruits de monstres, ambiances texturées, impacts mécaniques). Pour y parvenir, les concepteurs utilisent principalement deux approches technologiques :

1. **L'échantillonnage (Sampling)** : L'utilisation de fichiers audio enregistrés (.wav). Si cette technique offre un réalisme maximal, elle souffre d'un manque critique de flexibilité (le signal est figé) et d'une empreinte matérielle massive. Les banques de sons professionnelles saturent rapidement l'espace disque et la mémoire vive (RAM).
2. **La synthèse sonore** : Le son est généré de toutes pièces par des algorithmes mathématiques calculés à la volée par le processeur. Le plugin logiciel associé est extrêmement léger (quelques Mégaoctets) et offre une liberté totale de modulation en temps réel.

Le défi technologique actuel réside dans la **ré-synthèse inverse** : réussir à transformer automatiquement un échantillon sonore lourd et figé en un patch de synthèse léger, malléable et dynamique.

Fonctionnement d'un Synthétiseur Moderne et Choix Technologiques

Pour modéliser le comportement d'un synthétiseur par un réseau de neurones, il est nécessaire d'isoler les grandes méthodes de synthèse qui ont marqué l'évolution du traitement de signal :

- **La synthèse soustractive** : Consiste à filtrer des formes d'ondes riches en harmoniques (dent de scie, carré). C'est la méthode historique des premiers synthétiseurs analogiques.
- **La synthèse additive** : Reconstitue un spectre complexe en superposant des dizaines d'ondes sinusoïdales pures. Elle s'avère extrêmement lourde à paramétrer manuellement.
- **La synthèse FM (Modulation de Fréquence)** : Popularisée dans les années 80, elle repose sur la modulation de la fréquence d'une onde (porteuse) par une autre (modulatrice) à très haute vitesse. Elle excelle dans la création de textures métalliques, percussives et de timbres brillants.
- **La synthèse par Modélisation Physique** : Elle simule mathématiquement le comportement mécanique et acoustique d'instruments ou de résonateurs réels (cordes vibrantes, cavités, membranes).

Le choix de l'architecture hybride de *genesynth* :

Afin d'offrir une palette sonore à la fois riche et organique, ce projet couple la **Synthèse FM** et la **Modélisation Physique**. La FM génère l'excitation initiale et la brillance harmonique (l'impact), tandis qu'un résonateur physique (ligne de retard à boucle de rétroaction) apporte le comportement acoustique naturel.

Pour contrôler ce moteur, le système intègre des modules standards d'évolution temporelle (enveloppes ADSR dictant les phases d'Attaque, de Decay, de Sustain et de Release) et des outils de sculpture fréquentielle (filtres). L'objectif du réseau de neurones sera de prédire instantanément la configuration optimale de ces modules pour reproduire un son cible.

Problématique

Comment simplifier et optimiser la conception sonore en permettant à un utilisateur de recréer automatiquement un échantillon audio existant à l'aide d'un réseau de neurones déployé au sein d'un moteur de synthèse hybride polyphonique (FM + Guide d'ondes) ?

Public Cible

- **Les Sound Designers (Jeu vidéo, Cinéma)** : À la recherche d'un outil d'optimisation inverse capable de convertir un *sample* lourd en un modèle de synthèse physique ultra-léger et malléable.
- **Les Producteurs de Musique Électronique** : Souhaitant explorer de nouvelles textures sonores hybrides et générer des variations timbrales assistées par un algorithme d'apprentissage profond.

Objectifs Globaux

- **Développement d'un VST3 polyphonique** : Concevoir une bibliothèque dynamique native en C++ à l'aide du framework JUCE (limité à 8 voix simultanées), exécutable au sein d'un logiciel hôte (DAW).
- **Analyse de signal adaptée aux signaux tonaux** : Développer un pipeline d'analyse hybride temps-fréquence adapté aux signaux tonaux, combinant l'algorithme pYIN (estimation de la fréquence fondamentale) et l'extraction de l'enveloppe spectrale harmonique (banc de filtres et pics fréquentiels).
- **Régression par Deep Learning** : Entraîner un réseau de neurones léger sous Python (PyTorch), puis l'intégrer au VST via une bibliothèque d'inférence C++ native (RTNeural ou ONNX Runtime) pour prédire instantanément les 32 paramètres continus du moteur de synthèse hybride (FM étendue, Bruit, Filtre dynamique et Résonateur physique).

Objectifs Détaillés et Mesurables

Performances du Moteur Audio

- **Gestion déterministe de la mémoire : Strictement aucune allocation ni libération dynamique de mémoire (`new`, `malloc`, `free`)** au sein de la fonction principale de rendu audio (`processBlock`). La complexité algorithmique du moteur doit être en $O(1)$ pour proscrire toute gigue (*jitter*) ou micro-coupure numérique (*drop out*).
- **Respect du Buffer Budget** : Sous une configuration standard (tampon à 512 échantillons à 44.1 kHz, soit une fenêtre de calcul système de 11,6 ms), le calcul de la polyphonie maximale (8 voix simultanées combinant FM + Résonateur) doit s'exécuter en **moins de 2,5 ms** (consommation maximale de ~20% du thread audio).

Performance et Empreinte de l'IA

- **Temps d'inférence et d'analyse** : Pour un échantillon cible de 3 secondes, l'ensemble du pipeline (FFT + Inférence) doit s'exécuter en tâche de fond en **moins de 3 secondes** sur un processeur standard (sans assistance GPU).
- **Empreinte mémoire du modèle (RAM)** : L'allocation mémoire dédiée au stockage des poids du réseau de neurones ne doit pas excéder **15 Mo**.
- **Fuites de mémoire (Memory Leaks)** : Le delta de RAM consommée par l'application avant et après l'exécution d'une ré-synthèse doit être strictement égal à **0 octet**.

Temps de Chargement et Empreinte Mémoire Globale (Hors IA)

- **Initialisation et Rappel d'état (Recall)** : Le temps d'instanciation du plugin dans le DAW et le chargement d'un preset de synthèse complet doivent s'effectuer en **moins de 3 secondes**.
- **Empreinte RAM de base (Baseline Memory)** : L'allocation mémoire totale d'une instance vierge du plugin ne doit pas dépasser **60 Mo**.
- **Scalabilité par instance supplémentaire** : L'empreinte RAM incrémentale par instance supplémentaire ouverte dans un même projet doit être limitée à moins de **5 Mo** grâce au partage des ressources globales en mémoire (*Flyweight Pattern*).

Qualité de la Ré-synthèse et Traitement Python

- **Précision spectrale** : Atteindre un score de similarité spectrale moyen d'au moins **80 %** (mesuré via une distance Euclidienne sur les coefficients MFCC) sur le dataset de test.
- **Pipeline de prétraitement Python** : L'extraction des caractéristiques à partir des fichiers du dataset devra être automatisée via la bibliothèque librosa ou scipy. L'objectif est de

maintenir un temps de traitement global inférieur à **10 minutes** pour l'intégralité du dataset sur un processeur grand public.

Sécurité et Protection de la Propriété Intellectuelle (DRM)

- **Node-Locking local** : Le plugin doit intégrer un mécanisme de protection contre le partage non autorisé du binaire. Le système de vérification de licence doit être local et s'exécuter en moins de **100 ms** lors de l'initialisation du VST, sans requérir de connexion internet permanente (mode déconnecté).

Spécifications Fonctionnelles Détaillées

Description des Fonctionnalités du Plugin

Feature 1 : Module d'Entrée et Visualisation (Bandeau Inférieur)

- **Description utilisateur** : L'utilisateur clique sur le bouton *Imp.* (Import) pour charger un fichier audio, ou le glisse directement sur le widget *Sample* qui affiche la forme d'onde miniature du son cible à ré-synthétiser.
- **Spécifications Techniques** :
 - **Bouton Imp.** : Déclenche un `juce::FileChooser` natif filtré exclusivement pour les fichiers `.wav` monophoniques à 44.1 kHz.
 - **Widget Sample** : Instance de la classe `juce::AudioThumbnail` assurant un affichage asynchrone et fluide de la forme d'onde au sein du thread GUI (60 FPS).
 - **Composants d'Analyse C++ tiers** : L'analyse tonale en environnement natif s'appuiera sur la bibliothèque C++ `Aubio` (ou `Essentia`), liée de manière statique au projet JUCE et configurée en mode asynchrone hors du thread audio principal.
- **Pipeline d'Analyse Temporel/Fréquentiel (C++)** :
 - **Extraction de la Fondamentale (f_0)** : Le buffer de flottants issu du fichier audio importé est soumis à l'algorithme `aubio_pitch_yinfft` pour extraire la trajectoire temporelle de la hauteur tonale exacte.
 - **Calcul du Spectre Harmonique** : Une Transformée de Fourier Rapide via la classe native `juce::dsp::FFT` est exécutée sur des fenêtres de 2048 échantillons (Hop Size : 512) avec un fenêtrage de Hanning. Un algorithme de tracking extrait l'amplitude des 12 premiers partiels harmoniques synchrones en ciblant les indices complexes.
 - **Normalisation** : Les valeurs d'amplitudes spectrales sont converties sur une échelle logarithmique (Décibels) puis normalisées linéairement dans l'intervalle $[0.0, 1.0]$ pour correspondre au format du tenseur d'entrée attendu par le modèle ONNX.

Feature 2 : Potentiomètres de Conditionnement de l'IA (Macros)

Avant de lancer la génération, l'utilisateur dispose de 3 boutons rotatifs pour orienter ou contraindre les prédictions du modèle :

- **bright (Brillance)** : Agit comme un biais sur les hautes fréquences. Il force la couche finale de l'IA à privilégier un index de modulation FM élevé ou une ouverture maximale du filtre d'amortissement.

- **time (Temps / Durée)** : Définit la contrainte de longueur temporelle. Il applique un multiplicateur sur les paramètres d'enveloppe (ADSR et longueur de la ligne de retard du résonateur) pour forcer un son soit percussif, soit texturant.
- **spice (Mutation / Distorsion)** : Gère le taux d'injection de variables aléatoires (offset) dans l'espace latent ou le gain du soft-clipper, permettant d'apporter du "piment", de l'imprévisibilité ou de la saturation harmonique au patch généré.

Feature 3 : Moteur d'Inférence Déclenché

- **Description utilisateur** : Un clic sur le bouton *Generate* lance le calcul de ré-synthèse.
- **Spécifications Techniques** : Le bouton encapsule l'appel asynchrone au thread de l'IA (`juce::Thread`). Pendant l'inférence, le bouton passe visuellement en état "calcul" (grisé). Une fois le vecteur de la dernière couche calculé, les nœuds de l'espace de morphing central se déploient et s'animent pour refléter le nouveau patch.

Feature 4 : Sécurisation et Contrôle du Volume Final (Barregraphe)

- **Description utilisateur** : L'utilisateur dispose d'un potentiomètre de volume Master et d'un VU-mètre à LED pour surveiller le signal. Même si les réglages de résonance physique sont poussés à l'extrême, le système protège les haut-parleurs et les oreilles d'une coupure numérique agressive.
- **Spécifications Techniques** : Un composant GUI personnalisé basé sur `juce::Slider` et un timer rafraîchissant un barregraphe à 60 Hz mesurent la valeur RMS du bus master. Le signal est protégé en amont par le module de Soft-Cipping non-linéaire implanté en sortie du moteur DSP.

Feature 5 : Système de Validation de Licence Local

- **Description utilisateur** : Au premier démarrage, le plugin demande de saisir une clé de licence pour se déverrouiller définitivement.
- **Spécifications Techniques** :
 - **Génération du Machine ID** : Appel à `juce::SystemStats::getComputerUniqueIdentifier()` pour générer un hash SHA-256 unique basé sur les composants matériels de l'utilisateur (Node-locking).
 - **Algorithme de Vérification** : Le VST valide la clé entrée en vérifiant localement la signature cryptographique asymétrique (RSA via la bibliothèque légère `MbedTLS`) du Machine ID.
 - **Stockage de l'état** : L'état d'activation et la clé sont stockés de manière chiffrée via `juce::PropertiesFile`. Si l'activation échoue, le moteur audio est bypassé (silence total).

Feature 6 : Exploration Interactive et Sérendipité (Neural Morphing Pad)

- **Description utilisateur** : L'interface affiche une surface bidimensionnelle ou un graphe interactif représentant l'espace interne du réseau de neurones. En déplaçant le pointeur à la souris, l'utilisateur manipule directement les activations de la couche intermédiaire du modèle IA, permettant de "morpher" le son de manière organique soit pour corriger un timbre et le rendre plus réaliste, soit pour expérimenter et générer des textures étranges et inédites.
- **Spécifications Techniques** :
 - **Architecture Découplée** : Le modèle d'inférence est segmenté en deux étapes. Le Back-end (Bloc A) extrait un vecteur latent de taille 12 à 16 **neurones**. Le Front-end (Bloc B) mappe ce vecteur aux 32 paramètres et atténuateurs DSP du synthétiseur.
 - **Interaction Temps Réel** : Les coordonnées de friction ou de glissement (X, Y) de la souris sur le composant graphique JUCE (**mouseDrag**) viennent moduler dynamiquement les valeurs de ce vecteur latent intermédiaire à l'aide d'une matrice de projection continue.
 - **Inférence Dynamique Légère** : À chaque mouvement de souris, seule la dernière couche linéaire (Bloc B) du réseau est ré-exécutée sur le thread GUI. Ce calcul ultra-léger (coût d'inférence inférieur à 1 ms via *RTNeural*) garantit un retour sonore et visuel instantané sans bloquer le traitement audio.

Matrice MoSCoW du Projet

- **Must Have** : Moteur de synthèse hybride étendu à 32 paramètres (Double FM, Générateur de bruit coloré, Filtre Échelle, Résonateur physique), double enveloppe ADSR complète (Modulation et Amplitude), LFO, Gestionnaire de polyphonie (8 voix), Pipeline d'Analyse (pYIN + FFT Harmonique), Moteur d'Inférence IA.
- **Should Have** : Interface Interactive "Neural Morphing Pad" (Espace Latent étendu à 12-16 dimensions), Panneau de contrôle et d'édition Manuel (Bascule de Mode à 32 sliders/atténuateurs), Système de Validation de Licence Local (Node-locking), Potentiomètres macros (**spice, bright, time**).
- **Could Have** : Module de réverbération intégré (classe native **juce::Reverb** en bout de chaîne), Paramètre de contrôle du matériau (filtre passe-bas interne au feedback).
- **Won't Have** : Effets temporels complexes (delay synchrone, réverbération par convolution), Compatibilité multiplateforme exhaustive (compilation exclusive Windows 64-bit).

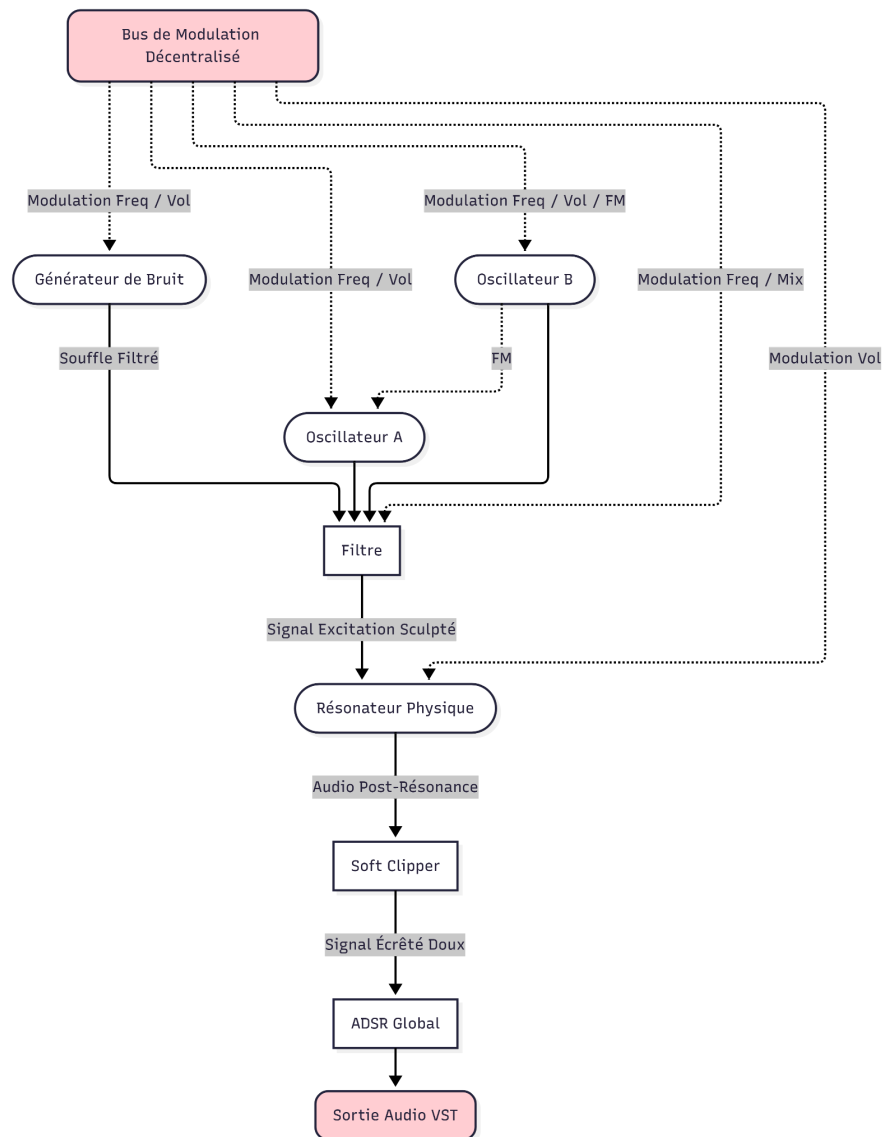
Architecture des Données, DSP et Stratégie IA

Architecture Algorithmique du Moteur de Synthèse (DSP C++)

Chaque voix de la polyphonie (8 voix simultanées gérées dynamiquement via la classe `juce::SynthesiserVoice`) encapsule l'architecture logicielle de traitement de signal. Le calcul complet est cadencé à une fréquence d'échantillonnage $f_s = 44100$ Hz.

Le Diagramme de Flux Audio et de Modulation

Le schéma suivant décrit l'acheminement précis du signal au sein d'une voix de synthèse ainsi que la topologie du bus de modulation décentralisé :



Le Générateur : Synthèse FM Étendue à 3 Opérateurs et Source de Bruit Modulable

Le module d'excitation génère des spectres harmoniques et inharmoniques complexes grâce à deux opérateurs modulateurs et un générateur de bruit à couleur et amplitude modulables.

- **Opérateurs FM (Carrier, Mod A, Mod B) :** Le moteur implémente une fonction de transfert de morphing géométrique continu notée $\Psi(\theta, w)$, où θ correspond à la phase instantanée et $w \in [0.0, 1.0]$ détermine la distorsion d'onde (0.0 = Sinus, 0.5 = Triangle/Dent de scie, 1.0 = Carré). Les volumes et fréquences de chaque opérateur sont dynamiquement assignables au bus de modulation via leurs atténuateurs respectifs.
- **Topologie Hybride Parallèle/Cascade :** L'oscillateur B possède un double routage asymétrique. Il module l'oscillateur A en fréquence (génération d'harmoniques FM complexes) mais injecte également son signal audio brut directement à l'entrée du filtre de manière parallèle, assurant une assise tonale riche en fréquences fondamentales (permettant par exemple de préserver le corps et la rondeur d'un son de basse sous l'impact FM).

- **Équation du Signal d'Excitation Global ($x_{\text{excitation}}[n]$) injecté dans le Filtre :**

Le signal entrant dans le filtre est la somme linéaire du bloc FM étendu et de la source de bruit modulée :

$$x_{\text{excitation}}[n] = x_{\text{fm}}[n] + (V_B[n] \cdot x_{\text{oscB}}[n]) + x_{\text{noise}}[n]$$

Où :

- $x_{\text{fm}}[n] = \Psi(\theta_c[n] + I_A[n] \cdot \Psi(\theta_{mA}[n], w_A), w_c)$ représente la composante de l'oscillateur A (générée par sa forme d'onde Ψ_A et modulée en fréquence par B).
 - $x_{\text{oscB}}[n] = \Psi(\theta_{mB}[n], w_B)$ correspond au signal audio brut généré de manière autonome par l'oscillateur B.
 - $V_B[n]$ est le gain d'envoi direct de l'oscillateur B vers le filtre, modulé dynamiquement.
 - $T_s = 1/f_s$, $\theta_i[n] = \theta_i[n-1] + 2\pi \cdot f_i \cdot T_s$, $f_c = f_{\text{midi}}$, $f_{mA} = \text{ratio}_A \cdot f_c$, et $f_{mB} = \text{ratio}_B \cdot f_c$.
- **Générateur de Bruit Modulable ($x_{\text{noise}}[n]$) :** Produit un bruit blanc algorithmique acheminé à travers un filtre passe-bas RIF à coefficient variable continu pour interpoler entre un spectre plat (Bruit Blanc, $w_{\text{color}} = 0.0$) et un spectre atténué à -3 dB/octave (Bruit Rose, $w_{\text{color}} = 1.0$). L'amplitude et la couleur spectrale du bruit répondent en temps réel aux enveloppes et LFO via leurs atténuateurs -/+ dédiés.

Le Module de Filtrage : Filtre Échelle (Ladder Filter)

Le signal combiné d'excitation ($x_{\text{excitation}}[n]$) traverse un filtre de type *Virtual Analog Ladder* à 4 pôles (pente de -24 dB/octave) émulant les circuits transistorisés classiques. La fréquence de coupure dynamique $F_c[n]$ et la résonance $R[n]$ traitent le signal avant son injection dans le résonateur.

Le Résonateur : Guide d'Ondes Numériques (Digital Waveguide)

Le signal filtré excite une structure de filtre en peigne à rétroaction modélisant une corde vibrante ou une cavité acoustique (Algorithme de Karplus-Strong étendu) :

- **Ligne de Retard (Delay Line)** : Un tampon circulaire géré par `juce::AudioBuffer<float>`. Sa taille en échantillons D est calculée via le paramètre de pitch indépendant du résonateur (f_{reso}) pour permettre des comportements inharmoniques, des percussions désaccordées ou des effets de désaxage volontaires par rapport au clavier :

$$D = f_s / f_{\text{reso}}.$$

- **Filtre d'Atténuation (Damping Filter)** : Un filtre passe-bas RIF à un seul coefficient placé dans la boucle pour simuler l'amortissement des hautes fréquences dans le matériau :

$$H(z) = w_{\text{damping}} + (1 - w_{\text{damping}})z^{-1}$$

- **Gain de Rétroaction (g)** : Coefficient linéaire $g \in [0.0, 0.999]$ déterminant la durée du sustain physique de la résonance.

Le Système de Sécurité et l'Enveloppe d'Amplitude Finale

- **Soft Clipper de Fin de Chaîne** : Placé strictement au bout de la chaîne de traitement (post-filtrage et post-résonance), le signal subit un tassement harmonique doux via une fonction de transfert tangente hyperbolique ($\tanh$) faisant office de limiteur de sécurité :

$$y_{\text{clipped}}[n] = \text{threshold} \cdot \tanh\left(\frac{y_{\text{waveguide}}[n]}{\text{threshold}}\right)$$

Cette fonction sature et arrondit de manière non-linéaire le signal dès qu'il approche du seuil critique, stabilisant le flux audio global sans coupure numérique agressive en cas de forte résonance ou d'auto-oscillation de la boucle physique.

- **Contrôle du Volume Final (ADSR Global)** : Le signal écrêté et stabilisé est multiplié en sortie ultime par la valeur d'amplitude instantanée générée par l'Enveloppe de Volume Globale ($A_{\text{global}}[n]$) avant d'être envoyé vers le bus master du DAW, dictant ainsi la dynamique temporelle et le silence final de la note.

Spécifications Détaillées du Moteur de Synthèse (Architecture DSP)

Pour couvrir la complexité acoustique des instruments réels (transitoires, souffles, résonances de matériaux) et la richesse des textures de synthèse modernes à la manière de solutions industrielles comme *Synplant*, le vecteur de sortie de l'IA est fixé à une dimension de **32 paramètres continus autonomes**. Ce choix d'architecture unifie l'apprentissage sous forme d'un problème de régression multivariée pure.

Formatage du Tenseur de Sortie (Labels du Modèle PyTorch)

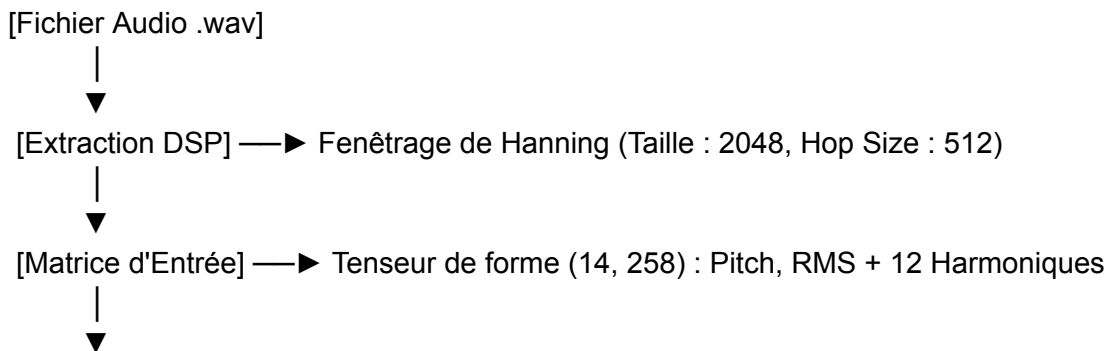
Le vecteur de taille (1, 32) généré par le modèle d'IA et mappé sur les composants JUCE suit la structure exacte suivante :

- **Module *osc. a* (4 paramètres) :**
 - **Output[0] : ratio** → Multiplicateur de la fréquence porteuse (0.5 à 12.0).
 - **Output[1] : wave** → Morphing continu de la forme d'onde (0.0 = Sinus à 1.0 = Onde complexe).
 - **Output[2] : Atténuateur -/+ vol** → Quantité de modulation sur le volume de l'oscillateur A.
 - **Output[3] : Atténuateur -/+ freq** → Quantité de modulation sur la fréquence (pitch) de l'oscillateur A.
- **Module *osc. b* (5 paramètres) :**
 - **Output[4] : ratio** → Multiplicateur de la fréquence modulatrice (0.5 à 12.0).
 - **Output[5] : wave** → Morphing continu de la forme d'onde (0.0 à 1.0).
 - **Output[6] : Atténuateur -/+ vol** → Quantité de modulation sur le volume de l'oscillateur B.
 - **Output[7] : Atténuateur -/+ freq** → Quantité de modulation sur la fréquence (pitch) de l'oscillateur B.
 - **Output[8] : Atténuateur -/+ FM** → Quantité de modulation sur l'index de modulation injecté vers l'oscillateur A.
- **Module *noise* (4 paramètres) :**
 - **Output[9] : vol** → Volume initial du générateur de bruit (0.0 à 1.0).
 - **Output[10] : freq** → Fréquence centrale de coupure/couleur du bruit (20 Hz à 20000 Hz).
 - **Output[11] : Atténuateur -/+ vol** → Quantité de modulation sur l'amplitude du souffle.
 - **Output[12] : Atténuateur -/+ freq** → Quantité de modulation sur la couleur du bruit.
- **Module *filter* (4 paramètres) :**
 - **Output[13] : type** → Fréquence de coupure initiale (Cutoff) du filtre Ladder -24 dB/oct.
 - **Output[14] : res.** → Taux de résonance du filtre (0.0 à 1.0).

- **Output[15]** : Atténuateur **-/+ freq** → Quantité de modulation sur la coupure du filtre.
- **Output[16]** : Atténuateur **-/+ mix** → Quantité de modulation sur la résonance du filtre.
- **Module **physic** (Résonateur - 5 paramètres) :**
 - **Output[17]** : **type** → Sélection/interpolation entre les modèles physiques (corde vibrante vs tube).
 - **Output[18]** : **input** → Point d'excitation virtuel sur le guide d'ondes (0.0 à 1.0).
 - **Output[19]** : **vol** → Volume initial/Gain de feedback (*g*) déterminant le sustain physique (0.0 à 0.999).
 - **Output[20]** : **freq** → Fréquence propre (accordage) du guide d'ondes (décalage de -24 à +24 demi-tons).
 - **Output[21]** : Atténuateur **-/+ vol** → Quantité de modulation sur la rétroaction/sustain physique.
- **Section Module de Modulation (6 paramètres) :**
 - **Output[22]** : Enveloppe Mod **A** → Temps d'attaque (1 ms à 3000 ms).
 - **Output[23]** : Env Mod **D** → Temps de déclin (10 ms à 5000 ms).
 - **Output[24]** : Env Mod **S** → Niveau de maintien (0.0 à 1.0).
 - **Output[25]** : Env Mod **R** → Temps de relâchement (10 ms à 5000 ms).
 - **Output[26]** : LFO **Hz** → Vitesse d'oscillation basse fréquence (0.1 Hz à 20 Hz).
 - **Output[27]** : LFO **amt** → Profondeur d'action globale du LFO (0.0 à 1.0).
- **Section Global Envelope (Amplitude Générale - 4 paramètres) :**
 - **Output[28]** : Global **A** → Temps d'attaque du volume principal.
 - **Output[29]** : Global **D** → Temps de déclin du volume principal.
 - **Output[30]** : Global **S** → Niveau de maintien du volume principal.
 - **Output[31]** : Global **R** → Temps de relâchement du volume principal.

Pipeline de Traitement et Tenseurs de l'IA

La ré-synthèse est traitée comme un problème de régression multivariée bornée. Le réseau de neurones n'agit pas directement sur l'audio temps réel mais détermine l'état statique initial et dynamique (modulations) du synthétiseur.



[Réseau de Neurones] —► Inférence C++ (ONNX Runtime / RTNeural) sur Background Thread



[Vecteur de Sortie] —► Tenseur unidimensionnel (1, 32) représentant les 32 paramètres cibles

- **Formatage du Tenseur d'Entrée (Features Tonaux)** : Pour capturer fidèlement la nature des signaux tonaux (sons de synthétiseurs, cloches, cordes), le pipeline d'analyse extrait un vecteur composite divisé en deux descripteurs sémantiques :
 - *Descripteur Temporel / Hauteur* : L'algorithme pYIN (ou YIN) suit l'évolution de la fréquence fondamentale (f_0) et de l'énergie efficace (RMS) sur 258 fenêtres temporelles.
 - *Descripteur Spectral / Harmonique* : Un calcul d'extraction de pics extrait la répartition de l'énergie des 12 premiers partiels harmoniques synchrones sur ces mêmes 258 fenêtres.
 - *Dimension finale du Tenseur d'Entrée* : Un tenseur bidimensionnel de forme (14, 258) (2 lignes pour le pitch/énergie + 12 lignes pour la structure des harmoniques, sur 258 fenêtres temporelles).
- **Formatage du Tenseur de Sortie (Labels)** : La couche de sortie du réseau utilise une fonction d'activation Sigmoidale pour contraindre les prédictions dans l'intervalle $[0.0, 1.0]$, qui sont ensuite redimensionnées par le moteur DSP selon les plages physiques réelles des 32 paramètres listés à la section 3.2.

Architecture Précise du Réseau de Neurones et Hyperparamètres

Pour permettre une exploration interactive innovante, l'architecture PyTorch est scindée en deux blocs distincts (Back-end et Front-end) afin d'exposer l'avant-dernière couche (l'Espace Latent) au contrôle de l'interface graphique en mode *Morphing*.

- **Modèle Global** : Perceptron Multicouches (MLP) ou Réseau Convolutif 1D (1D-CNN) profond.
- **Couche d'Entrée (Input Layer)** : Aplatissement (Flattening) de la matrice harmonique de taille (14, 258), soit un vecteur linéaire de 3612 neurones.
- **Bloc A - Couches Cachées et Espace Latent (Back-end)** :
 - **Dense 1** : 1024 neurones + ReLU + Batch Normalization + Dropout (rate = 0.2).
 - **Dense 2** : 512 neurones + ReLU + Batch Normalization.
 - **Dense 3** : 256 neurones + ReLU + Batch Normalization.
 - **Dense 4 (Goulot d'étranglement / Espace Latent)** : 12 à 16 neurones. Ce dimensionnement étendu est spécifiquement calibré pour capturer les régimes chaotiques et hautement sensibles du modèle physique (Digital Waveguide) sans lisser ou uniformiser les accidents acoustiques. Cet espace est directement connecté aux surfaces de contrôle géométriques de l'interface utilisateur.

- **Bloc B - Couche de Sortie de Régression (Front-end) :**
 - 32 **neurones** (un par paramètre DSP et atténuateur du moteur) avec fonction d'activation Sigmoidale, connectée directement à l'espace latent de la couche précédente.

Philosophie du Morphing Pad et Objectifs Ergonomiques

L'accès direct aux 12-16 dimensions de l'espace latent via l'interface graphique répond à une double philosophie d'utilisation, validée par prototypage fonctionnel :

- **Algorithme d'Alignement et de Restitution :** Permettre à l'opérateur de stabiliser, calibrer et "rattraper" un patch prédit par l'IA afin de maximiser son esthétique ou son réalisme acoustique par rapport au modèle d'origine (ajustement fin autour des configurations stables).
- **Espace d'Exploration et de Sérendipité :** Exploiter la topologie de l'espace compressé pour stimuler la créativité par l'expérimentation pure. Les zones frontières de l'espace latent, bien que structurellement instables, imprévisibles ou inharmoniques, sont volontairement conservées afin de permettre l'émergence de textures sonores complexes, inédites et transposables musicalement (l'imprévisibilité physique étant traitée comme un attribut créatif).

Protocole d'Apprentissage par Curriculum Entrelacé Évolutif

Dans le but d'éviter que le modèle ne s'enferme dans des minima locaux ou ne subisse un phénomène d'oubli catastrophique (*Catastrophic Forgetting*) lié à une transition trop brutale entre signaux purement synthétiques (FM) et physiques (Guide d'ondes), le protocole d'apprentissage implémente une stratégie de **Curriculum Aléatoire Entrelacé (Interleaved Curriculum Learning)**.

Le jeu de données est segmenté selon la nature topologique des patches (signaux purement FM/Noise vs signaux couplés au résonateur). La distribution des mini-batches évolue dynamiquement au cours des époques selon trois paliers :

1. **Palier d'Amorçage (Époques 1 à 30) :** Les batches sont constitués de 80 % de signaux d'excitation simples (résonateur désactivé) et 20 % de signaux résonnants complets. Le réseau structure ses premières couches sur les composantes macroscopiques du timbre sans saturer son gradient.
2. **Palier de Transition (Époques 31 à 70) :** Un équilibre statistique strict à 50 % / 50 % est introduit pour forcer le modèle à cartographier les interactions et corrélations entre les modulations FM et la ligne de retard.
3. **Palier de Verrouillage Asymétrique (Époques 71 à 150) :** La proportion bascule à 20 % de sons simples et 80 % de sons hautement résonnants. C'est lors de ce jalon que la fonction de perte bascule sur la **Weighted MSE Loss**, appliquant une pénalité sévère

sur le gain de feedback (facteur 5.0 sur [Output\[19\]](#)) et le pitch propre du guide d'ondes (facteur 4.0 sur [Output\[20\]](#)).

- **Optimiseur** : AdamW (Learning Rate = 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$, Weight Decay = 10^{-4}).
- **Répartition du Dataset** : 80% Train (8000 samples), 10% Validation (1000 samples), 10% Test (1000 samples).

Validation par Presets d'Ancrage et Phase d'Alignement Musical

À l'issue de la phase d'apprentissage généralisé, le modèle de ré-synthèse subit un protocole de calibration comparative face à une banque de "Presets d'Ancrage" (Anchor Presets). Cette banque, éditée manuellement par un opérateur au sein de l'UI modulaire, modélise les architectures cibles de timbres standards et communs de l'industrie (Piano, Rhodes, instruments à vent, basses soustractives).

Le signal audio de ces références est soumis au modèle afin de mesurer l'erreur de transfert spectral (distance MFCC). En cas de dérive ou d'ambiguïté dans l'espace des paramètres (plusieurs configurations de curseurs menant au même profil harmonique), une phase de **Fine-Tuning à très faible taux d'apprentissage** ($LR = 10^{-5}$) est instanciée sur ce sous-ensemble de données communes. Ce processus d'alignement garantit que l'espace latent de l'IA converge en priorité vers des configurations de curseurs musicalement cohérentes et exploitables pour les architectures sonores les plus usuelles.

Protocole de Génération Automatique du Dataset

Un script autonome écrit en Python ([scipy.io.wavfile](#) + [librosa](#)) génère de manière déterministe les couples (Audio / Paramètres) en faisant varier de manière pseudo-aléatoire (LHS - *Latin Hypercube Sampling*) les 32 variables cibles du moteur DSP.

- **Format des données** : Format PCM non compressé ([.wav](#)), Monophonique, 16-bit, 44.1 kHz. Un échantillon unique de 3 secondes pèse précisément 264,6 Ko.
- **Volume et Stockage** : Le jeu de données complet de 10000 échantillons représente une empreinte disque totale maîtrisée de **2,64 Go**, permettant un stockage et un entraînement local complet et rapide sur la station de travail de R&D.
- **Augmentation de Données (Data Augmentation)** : Afin de garantir la portabilité et la robustesse de l'IA face à de vrais instruments enregistrés par microphone (biais de domaine), le script de génération injecte des micro-décalages de phase, des filtrages variables et un bruit de fond rose de calibration de faible intensité aux signaux synthétisés, immunisant le réseau contre les artefacts extérieurs.

Calendrier de Réalisation et Planification des Sprints (3 Mois)

Le projet est structuré selon la méthodologie Agile, divisé en 6 Sprints de 14 jours. Chaque sprint correspond à un jalon technique précis et mesurable.

Semaines 1-2 : Moteur Audio Polyphonique DSP (C++)

- **Objectif** : Obtenir l'architecture de synthèse fonctionnelle sans IA.
- **Livrables** : Un binaire VST3 préliminaire capable de générer des sons FM résonnants en temps réel lorsqu'il est joué au clavier MIDI, sans craquement ni allocation dynamique dans le thread audio.

Semaines 3-4 : Pipeline de Données et Dataset (Python)

- **Objectif** : Modéliser l'extracteur de caractéristiques et exporter les données d'entraînement.
- **Livrables** : Un dataset d'entraînement complet de **10 000 échantillons .wav mono** (2,64 Go) et de leurs fichiers métadonnées .json, extrait via librosa.pyin() et librosa.effects.harmonic().

Semaines 5-6 : Architecture et Entraînement de l'IA (Python)

- **Objectif** : Concevoir le modèle de Deep Learning et atteindre les critères de convergence.
- **Livrables** : Fichiers de poids du réseau de neurones (.onnx) entraînés sous PyTorch et validés scientifiquement à $\geq 80\%$ de similarité spectrale.

Semaines 7-8 : Intégration de l'IA et Multi-threading (C++)

- **Objectif** : Faire tourner l'inférence de l'IA à l'intérieur du plugin VST.
- **Livrables** : Configuration de l'environnement C++ pour l'inclusion de la bibliothèque Aubio/Essentia et de ONNX Runtime. Le VST3 lance l'IA en tâche de fond de manière thread-safe via l'APVTS.

Sprint 5 (Semaines 9-10) : Interface Graphique Interactive et Sécurité

- **Objectif** : Rendre le plugin fonctionnel et interactif selon les maquettes graphiques.
- **Livrables** : Composant personnalisé **Neural Morphing Pad** connecté à l'espace latent en C++. Intégration des boutons macros et du système de licence par Node-locking.

Sprint 6 (Semaines 11-12) : Recette, Optimisation et Rapport Final

- **Objectif** : Valider la stabilité industrielle du produit et préparer la soutenance.
- **Livrables** : Produit logiciel figé en version stable v1.0.0 (profilage CPU validé à $\leq 2,5 \text{ ms}$ par bloc, 0 fuite mémoire), accompagné du mémoire de fin d'études.

Spécifications de l'Interface Utilisateur (GUI) et Maquettage

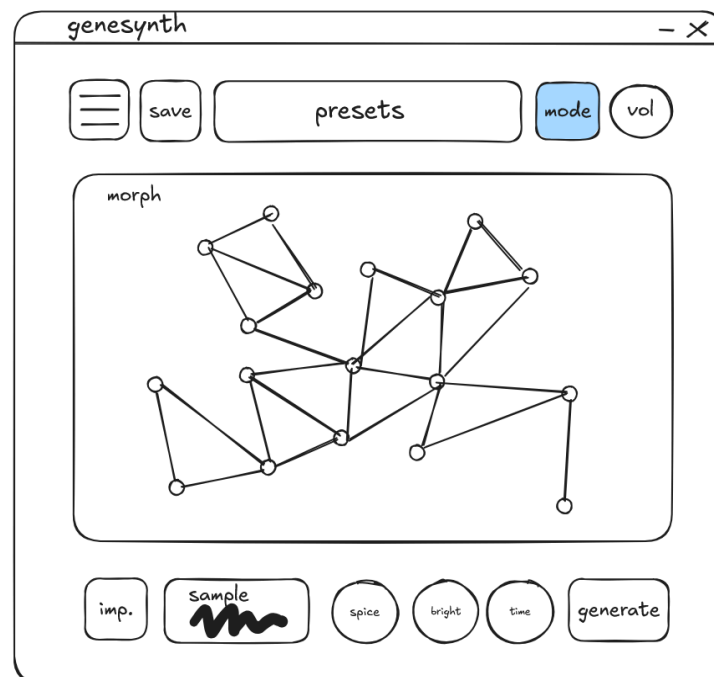
Charte Graphique et Environnement Visuel

- **Dimensions de la fenêtre** : Taille fixe de 800×500 pixels (non redimensionnable pour garantir l'intégrité du layout).
- **Esthétique globale** : Courant Minimaliste Vectoriel / Cyberpunk Épuré. Pas de textures complexes, utilisation de lignes nettes et d'aplats de couleurs de type *Flat Design*.
- **Palette de couleurs (Hexadécimal)** :
 - Fond principal (Background) : #1E222B (Anthracite type "Dark Mode").
 - Éléments interactifs actifs (Graphe, Sliders de fond) : #528BFF (Bleu néon).
 - Indicateurs IA / Validation / Activation : #28C76F (Vert émeraude).
 - Textes et Typographie : #ECEFF1 (Blanc cassé à haut contraste). Typographie monospacée (type *Roboto Mono*).

Découpage Panoramique de l'Interface (Layout)

Panneau Simplifié

L'écran est structuré en trois zones horizontales distinctes via un agencement basé sur `juce::FlexBox` :



1. Le Panneau Supérieur (Top Panel - Zone de Gestion) :

- Regroupe les boutons système discrets : Logo du plugin genesynth, Menu, boutons save, gestionnaire de presets, sélecteur de mode et potentiomètre de volume Master global (vol).

2. Le Panneau Central (Main Body - Zone interactive Morph) :

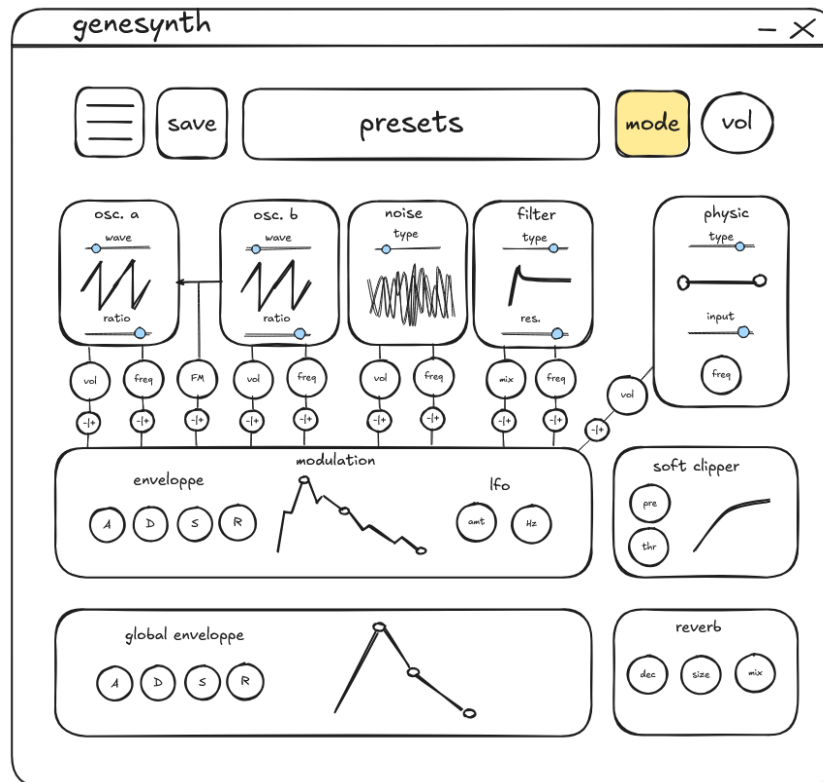
- Occupe 70% de la hauteur de la fenêtre. Il affiche exclusivement le **Neural Morphing Pad** : une constellation de cercles interconnectés par des lignes dynamiques dessinées via l'API juce::Graphics. Chaque cercle représente un nœud de l'espace latent.

3. Le Panneau Inférieur (Bottom Panel - Zone de Contrôle et Workflow) :

- Héberge à gauche le module d'acquisition (bouton d'importation imp. et le widget visuel sample).
- Héberge au centre les trois potentiomètres rotatifs macros de conditionnement : spice, bright et time.
- Héberge à droite le gros bouton d'action principal #28C76F intitulé generate.

Le Panneau Central en Mode Synthétiseur Manuel

Lorsque l'utilisateur bascule en mode Manuel, l'interface graphique de *genesynth* se structure en 5 sous-modules indépendants connectés à une section de modulation centralisée en bas de l'écran, reproduisant fidèlement l'ergonomie des systèmes modulaires de type Eurorack



1. Description Structurelle des Blocs UI

- **Modules osc. a & osc. b** : Intègrent un afficheur vectoriel de la forme d'onde. Ils disposent d'un bouton rotatif vertical pour le **ratio**, d'une glissière horizontale pour la forme d'onde (**wave**) et de deux potentiomètres de modulation bipolaires (**-/+ vol** et **-/+ freq**). Le module B dispose d'un troisième atténuateur bipolaire pour l'index **FM**.
- **Module noise** : Affiche une représentation visuelle du souffle. Il comporte un potentiomètre rotatif vertical pour le volume de base (**vol**), un curseur horizontal pour la couleur spectrale du bruit (**type**) et deux atténuateurs de modulation bipolaires (**-/+ vol** et **-/+ freq**).
- **Module filter** : Affiche la courbe de réponse de coupure. Il comprend un potentiomètre rotatif vertical pour la fréquence de coupure (**type**), un curseur horizontal pour la résonance (**res.**) et deux atténuateurs bipolaires de modulation (**-/+ freq** et **-/+ mix**).
- **Module physic (Résonateur)** : Affiche le comportement d'amortissement du modèle physique. Il est constitué d'un potentiomètre rotatif vertical pour le volume de rétroaction (**vol**), d'un sélecteur de comportement (**type**), d'un curseur horizontal pour le point d'impact (**input**), d'un potentiomètre pour l'accordage (**freq**) et d'un atténuateur bipolaire de modulation (**-/+ vol**).

2. Logique Mathématique d'Atténuation Temporaire (Régulation des Bornes)

Pour l'ensemble des atténuateurs bipolaires **-/+** de l'interface, le comportement au sein du thread audio C++ implémente une logique de restriction de plage dynamique proportionnelle. Conformément aux exigences ergonomiques, la valeur instantanée finale d'un composant (V_{final}) calculée à chaque échantillon par le signal combiné de modulation ($M \in [0.0, 1.0]$) répond à l'équation suivante :

$$V_{\text{final}} = V_{\text{principal}} + (M \cdot (A_{\text{attenuateur}} - V_{\text{principal}}))$$

Où $V_{\text{principal}}$ représente la valeur du bouton principal (la borne d'origine) et $A_{\text{attenuateur}}$ la valeur du potentiomètre de modulation **-/+** faisant office de borne dynamique supérieure ou inférieure.

Cinématique de l'Application (Parcours Utilisateur)

Le flux de travail global de *genesynth* suit une cinématique linéaire en "U" (Bandeau inférieur → Validation → Zone centrale), complétée par une boucle d'édition avancée (Bascule de Mode) pour les sound designers.

- **Étape 1 : Acquisition (Bandeau Bas - Gauche)** : L'utilisateur charge son fichier *.wav* cible via le bouton *imp.*. Le widget *sample* dessine instantanément la forme d'onde miniature pour valider visuellement la bonne lecture du signal audio en mémoire RAM.
- **Étape 2 : Conditionnement (Bandeau Bas - Centre)** : L'utilisateur ajuste les trois potentiomètres macros *spice*, *bright* et *time* pour appliquer des contraintes esthétiques et donner une direction artistique à la génération de l'IA.
- **Étape 3 : Inférence Asynchrone (Bandeau Bas - Droite)** : L'utilisateur clique sur le bouton vert *generate*. L'UI se verrouille partiellement. Un *juce::Thread* secondaire s'exécute en arrière-plan : il extrait la hauteur tonale fondamentale et le profil harmonique du signal, puis exécute le Bloc A du modèle de Deep Learning. Le thread audio temps réel continue de traiter le signal sans aucun glitch ou saut de buffer (*Buffer Underrun*).
- **Étape 4 : Exploration Initiale (Zone Centrale - Mode Morph)** : Le thread secondaire renvoie le vecteur intermédiaire au thread GUI. Le graphe du *Neural Morphing Pad* s'illumine. L'utilisateur clique et glisse sa souris sur la constellation de nœuds. Chaque micro-déplacement applique un offset dans l'espace latent et ré-exécute instantanément le Bloc B du réseau (l'inférence prend moins de *0.5 ms*), mettant à jour les 32 paramètres de synthèse en temps réel de manière fluide et organique.
- **Étape 5 : Bifurcation et Peaufinage Étendue (Zone Centrale - Mode Manuel)** : Si l'utilisateur souhaite sculpter le timbre au-delà des prédictions de l'IA, il clique sur le commutateur *mode*. La constellation s'efface en fondu. L'interface affiche le panneau modulaire complet (les 32 sliders et atténuateurs de modulation calés sur la position exacte prédite par l'IA). L'utilisateur ajuste à la main les ratios FM, injecte du souffle (*noise*), modifie les temps ADSR ou module le filtre via le LFO pour finaliser l'identité sonore de son patch.
- **Étape 6 : Sauvegarde et Export (Bandeau Supérieur)** : Une fois la texture idéale stabilisée (que ce soit en mode Morph ou après édition manuelle), l'utilisateur clique sur le bouton *save* du panneau supérieur pour enregistrer son patch nommé au format natif dans le gestionnaire de *presets* du VST.

Calendrier de Réalisation et Planification des Sprints (3 Mois)

Le projet est structuré selon la méthodologie Agile, divisé en 6 Sprints de 14 jours. Chaque sprint correspond à un jalon technique précis et mesurable.

Semaines 1-2 : Moteur Audio Polyphonique DSP (C++)

- **Objectif** : Obtenir l'architecture de synthèse fonctionnelle sans IA.
- **Livrables** : Un binaire VST3 préliminaire capable de générer des sons FM résonnants en temps réel lorsqu'il est joué au clavier MIDI, sans craquement ni allocation dynamique dans le thread audio.

Semaines 3-4 : Pipeline de Données et Dataset (Python)

- **Objectif** : Modéliser l'extracteur de caractéristiques et exporter les données d'entraînement.
- **Livrables** : Un dataset d'entraînement complet de **10 000 échantillons .wav mono** (2,64 Go) et de leurs fichiers métadonnées .json, extrait via librosa.pyin() et librosa.effects.harmonic().

Semaines 5-6 : Architecture et Entraînement de l'IA (Python)

- **Objectif** : Concevoir le modèle de Deep Learning et atteindre les critères de convergence.
- **Livrables** : Fichiers de poids du réseau de neurones (.onnx) entraînés sous PyTorch et validés scientifiquement à $\geq 80\%$ de similarité spectrale.

Semaines 7-8 : Intégration de l'IA et Multi-threading (C++)

- **Objectif** : Faire tourner l'inférence de l'IA à l'intérieur du plugin VST.
- **Livrables** : Configuration de l'environnement C++ pour l'inclusion de la bibliothèque Aubio/Essentia et de ONNX Runtime. Le VST3 lance l'IA en tâche de fond de manière thread-safe via l'APVTS.

Sprint 5 (Semaines 9-10) : Interface Graphique Interactive et Sécurité

- **Objectif** : Rendre le plugin fonctionnel et interactif selon les maquettes graphiques.
- **Livrables** : Composant personnalisé **Neural Morphing Pad** connecté à l'espace latent en C++. Intégration des boutons macros et du système de licence par Node-locking.

Sprint 6 (Semaines 11-12) : Recette, Optimisation et Rapport Final

- **Objectif** : Valider la stabilité industrielle du produit et préparer la soutenance.
- **Livrables** : Produit logiciel figé en version stable v1.0.0 (profilage CPU validé à $\leq 2,5 \text{ ms}$ par bloc, 0 fuite mémoire), accompagné du mémoire de fin d'études.

Plan de Validation et de Test

Chaque composant fera l'objet d'une validation rigoureuse :

Tests Unitaires Logiciels (Algorithmes DSP et Sécurité)

Identifiant	Composant Testé	Méthode de Validation	Résultat Attendu
TEST-DSP-01	Oscillateur FM	Injection de paramètres statiques ($A_c = 1, I = 0$). Mesure du signal de sortie.	Génération d'une onde sinusoïdale pure parfaite, sans distorsion.
TEST-DSP-02	Soft Clipper	Injection d'un signal continu d'amplitude $A = 2.0$ (saturation théorique).	Le signal de sortie doit être strictement bridé et arrondi à un maximum de ≤ 1.0 .
TEST-DSP-03	Thread Audio	Mesure de l'allocation mémoire via un outil de traçage pendant 5 min de jeu.	0 octet d'allocation sur le tas (<i>heap</i>) détecté. Aucune fuite mémoire.
TEST-SEC-01	Node-Locking	Modification manuelle d'un octet de l'identifiant machine dans le fichier de config.	Le plugin doit refuser de démarrer au cycle suivant et basculer en mode Locked (Mute).

Tests d'Intégration et de Performance de l'IA

Identifiant	Composant Testé	Méthode de Validation	Résultat Attendu
TEST-IA-01	Stabilité UI / Audio	Déclenchement de l'inférence IA pendant qu'on joue de lourds accords au clavier maître.	Le thread audio continue de jouer à 44.1 kHz sans coupure, l'UI reste réactive à 60 FPS.
TEST-IA-02	Précision Régression	Comparaison des paramètres prédits par l'IA par rapport aux paramètres réels du jeu de test.	L'erreur quadratique moyenne (MSE) par paramètre doit être inférieure à 0.05 (précision de 95%).
TEST-IA-03	Fluidité Morphing	Balayage rapide de la souris sur le <i>Morph Pad</i> central pendant 10 secondes.	Le Bloc B du réseau recalcule les curseurs en moins de 1 ms , aucun lag audio ou visuel.

Gestion des Risques et Plans de Secours

- **Risque 1 : Difficultés d'intégration ou instabilité de la bibliothèque d'inférence (ONNX/RTNeural) en C++.**
 - *Plan de secours* : Si l'intégration native prend trop de temps, le modèle d'IA sera exécuté au sein d'un script Python local indépendant, et le VST JUCE communiquera avec lui via un protocole de communication locale léger (Sockets TCP).
- **Risque 2 : Instabilité mathématique du résonateur (Feedback Loop) malgré le Soft Clipper.**
 - *Plan de secours* : Si la boucle de rétroaction s'avère trop instable pour le réseau de neurones lors des phases de test, le moteur audio basculera sur une banque classique de filtres passe-bande parallèles sans feedback, plus prévisible pour l'IA.
- **Risque 3 : Difficultés de liaison (linking) ou de cross-compilation des bibliothèques d'analyse tierces (Aubio/Essentia) dans l'environnement JUCE.**
 - *Description* : La compilation de bibliothèques C++ externes peut générer des conflits d'architectures. Ce point est hautement amené à évoluer ou changer durant la phase de prototypage.
 - *Plan de secours* : Si l'intégration d'Aubio/Essentia bloque le développement, l'extraction de la fondamentale (f_0) basculera sur un algorithme d'**Autocorrélation Temporelle Directe** codé manuellement en C++ natif pur (sans dépendance externe). L'extraction des 12 pics harmoniques se fera ensuite via un simple algorithme de recherche de maximums locaux (*Peak Picking*) autour des multiples entiers de la fondamentale calculée sur la `juce::dsp::FFT`.

Glossaire

- **DAW (Digital Audio Workstation)** : Le logiciel hôte (ex: Ableton Live, FL Studio) qui charge le VST et gère le thread audio principal.
- **VST3 (Virtual Studio Technology v3)** : Format de bibliothèque dynamique standard de l'industrie audio développé par Steinberg, implémenté ici via JUCE.
- **FFT / MFCC** : Outils mathématiques permettant de traduire un fichier audio brut (temporel) en une empreinte de fréquences (spectrale) compréhensible par un réseau de neurones.
- **Digital Waveguide** : Modèle physique de guide d'ondes qui simule la propagation et la réflexion des ondes acoustiques à l'aide de lignes de retard et de filtres.
- **Espace Latent (Couche Cachée)** : Représentation compressée de l'information au sein d'un réseau de neurones, capturant les caractéristiques sémantiques ou timbrales essentielles des données.